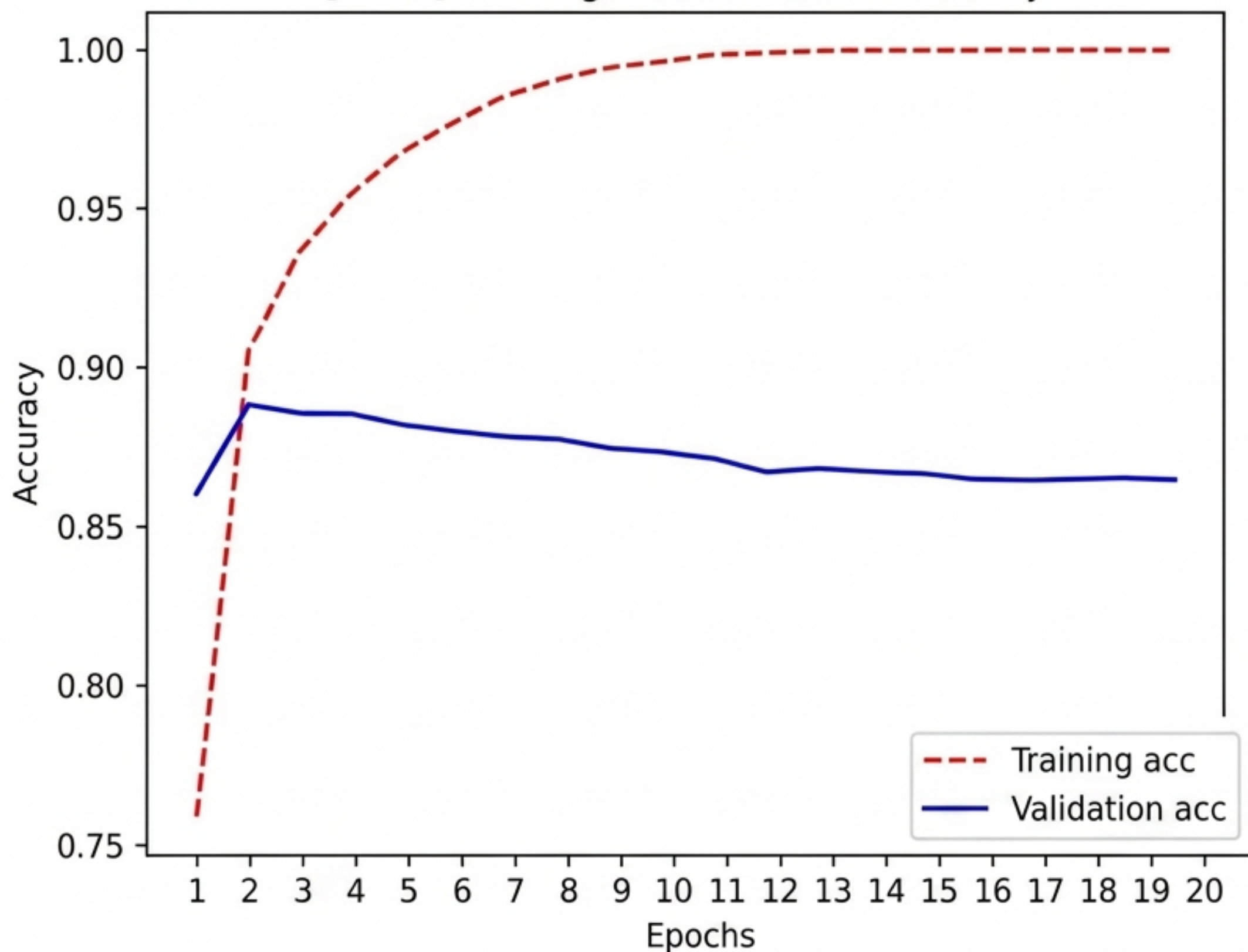


[IMDB] Training and validation accuracy



Hình 4.5: Sự biến thiên của Accuracy

Đánh giá Huấn luyện: Biểu đồ Accuracy

Training Accuracy (chấm bi):

Tiến sát mức hoàn hảo 100% (1.0).

Validation Accuracy (đường liền):

Đạt đỉnh khoảng **88%** ngay tại epoch thứ 4, sau đó bắt đầu giảm dần.

Hiện tượng & Giải pháp:

Quá khớp (Overfitting) rõ rệt sau epoch 4. Cần áp dụng **Early Stopping** bằng cách huấn luyện lại mô hình mới và chốt tại epochs=4 để lấy kết quả tối ưu nhất.

Dự đoán Dữ liệu Mới (Predictions)

Sử dụng hàm **predict()** để đánh giá dữ liệu mới ngoài thực tế.

```
predictions = model.predict(x_test)
print(predictions)

# array([[ 0.98006207] ,
#        [ 0.99758697] ,
#        ...,
#        [ 0.02885115] ,
#        [ 0.65371346]], dtype=float32)
```





Bài toán Reuters: Phân loại Đa lớp (Multiclass Classification)

Mục tiêu: Phân loại các bản tin tức của hãng thông tấn Reuters vào 46 chủ đề khác nhau.

Đặc trưng

Single-label, Multiclass
(Mỗi bài báo chỉ thuộc duy nhất 1 chủ đề trong 46 lựa chọn).

Kích thước Dữ liệu

8,982 mẫu huấn luyện (Train) & 2,246 mẫu kiểm tra (Test).

Tiền xử lý

Sử dụng Multi-hot Encoding cho dữ liệu đầu vào text tương tự như bài toán IMDb.

Mã hóa Nhãn: One-Hot Encoding

Khác với IMDb chỉ có 2 nhãn (0, 1) dùng số nguyên, Reuters có tới 46 chủ đề.

Cần biến đổi mảng nhãn thành dạng Vector (**Mã hóa phân loại**).

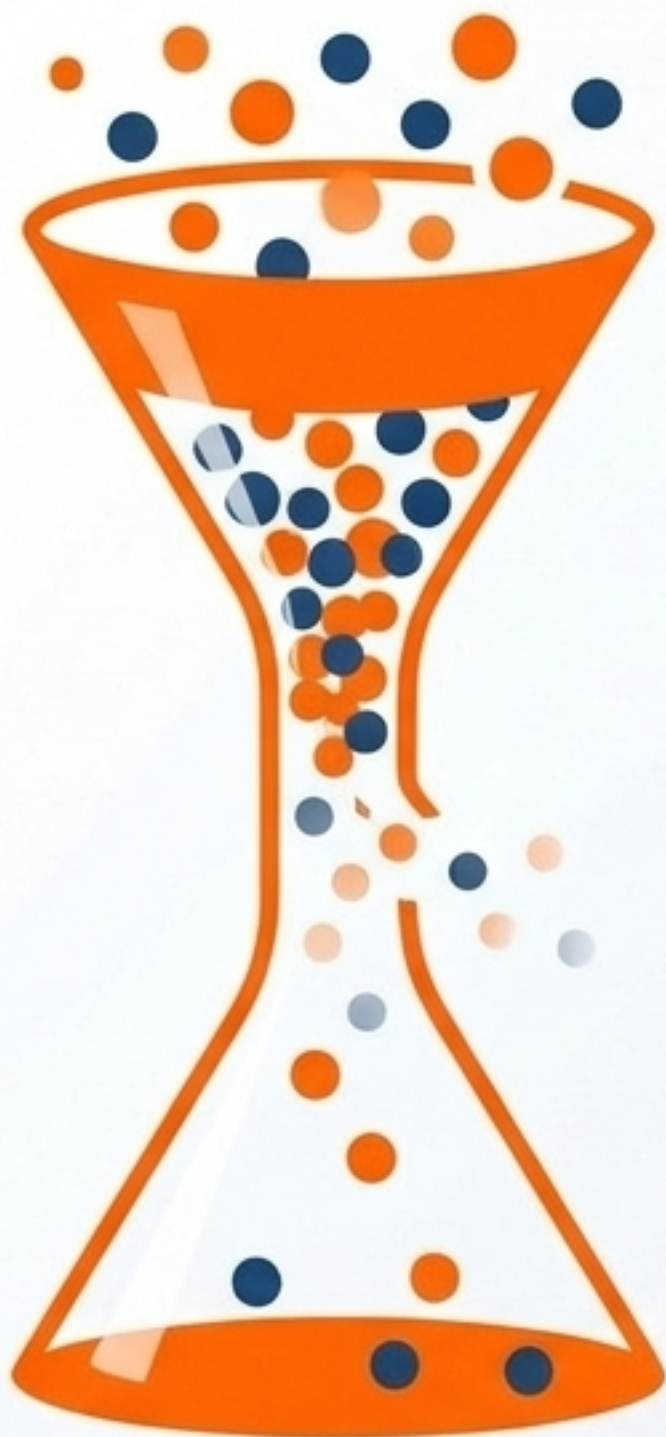
```
from keras.utils import to_categorical
```

```
# Chuyển [3, 10, 45, ...] thành ma trận 46 cột (0 và 1)  
y_train = to_categorical(train_labels)  
y_test = to_categorical(test_labels)
```



Ví dụ: `y_train[0]` có nhãn 3 -> Sẽ được chuyển thành một vector gồm 45 số 0 và duy nhất số 1 tại index 3.

Kiến trúc Mô hình & Nút thắt Cổ chai Thông tin



- **Thắt cổ chai thông tin (Information Bottleneck):** Lớp Dense 16 units là quá nhỏ để truyền tải 46 chủ đề, gây mất mát thông tin vĩnh viễn. Bắt buộc tăng lên 64 units.
- **Lớp đầu ra (Output Layer):** 46 units đại diện cho 46 chủ đề, kết hợp hàm kích hoạt Softmax.

```
model = keras.Sequential([  
    layers.Dense(64, activation="relu"),  
    layers.Dense(64, activation="relu"),  
    layers.Dense(46, activation="softmax")  
])
```

Biên dịch Mô hình Đa lớp



Softmax

Xuất ra một phân phối xác suất gồm 46 chiều (Tổng xác suất luôn bằng 1).

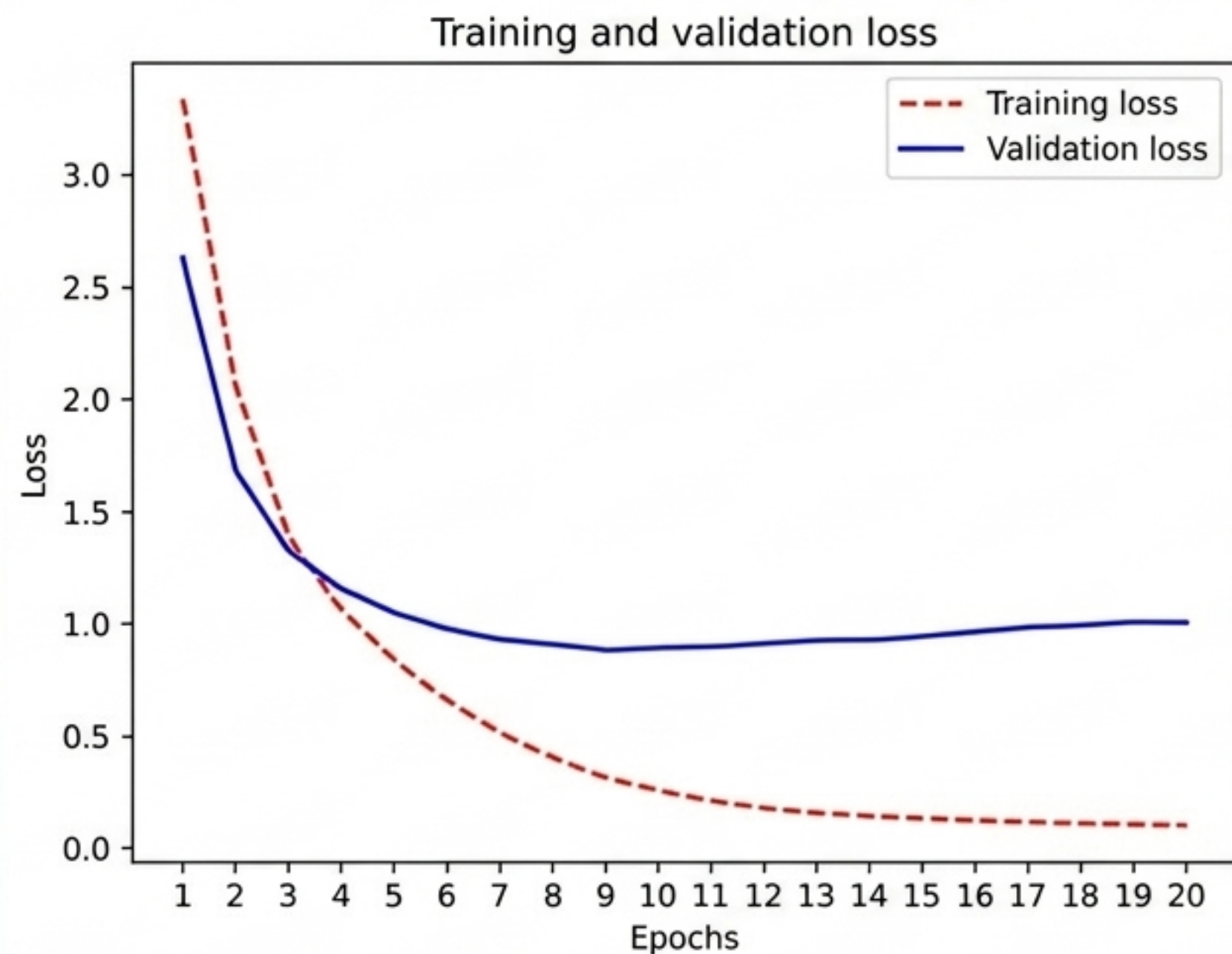
Categorical Crossentropy

Hàm mất mát chuyên dụng để đo lường khoảng cách giữa 2 phân phối xác suất đa chiều.

```
model.compile(optimizer="adam",  
              loss="categorical_crossentropy",  
              metrics=["accuracy"])
```

```
history = model.fit(x_train, y_train,  
                   epochs=20, batch_size=512,  
                   validation_split=0.2)
```

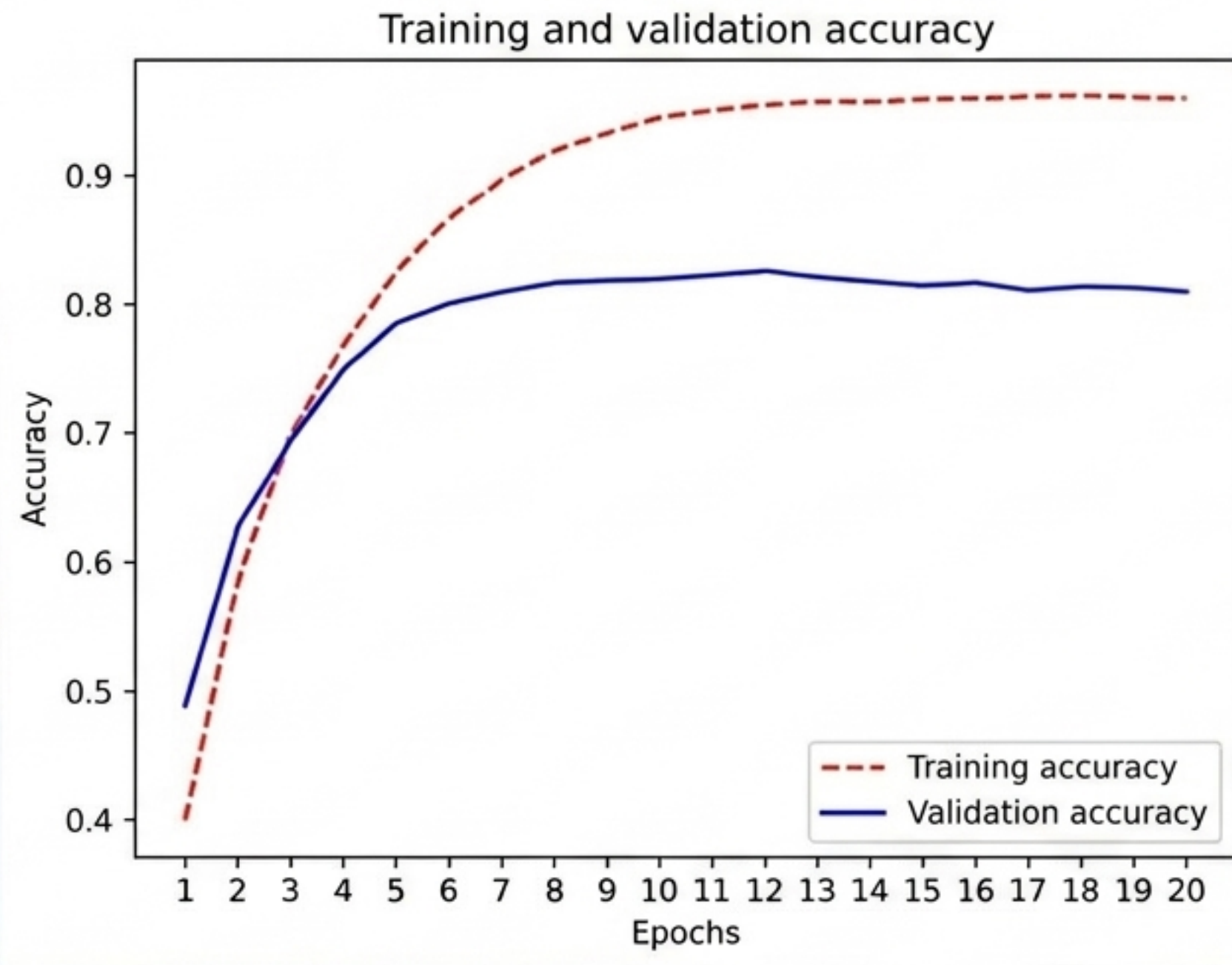
Hàm mất mát & Biểu đồ Loss



Hình 4.6: Training và Validation Loss (Reuters)

- **Đo lường:** Hàm Loss đo lường sự sai lệch giữa dự đoán và thực tế.
- **Hiện tượng Overfitting:** Xảy ra ở hầu như mọi bài toán Học sâu.
- **Điểm bùng phát:** Loss trên tập validation (đường xanh) giảm dần nhưng bắt đầu tăng mạnh và đảo chiều sau epoch thứ 9.

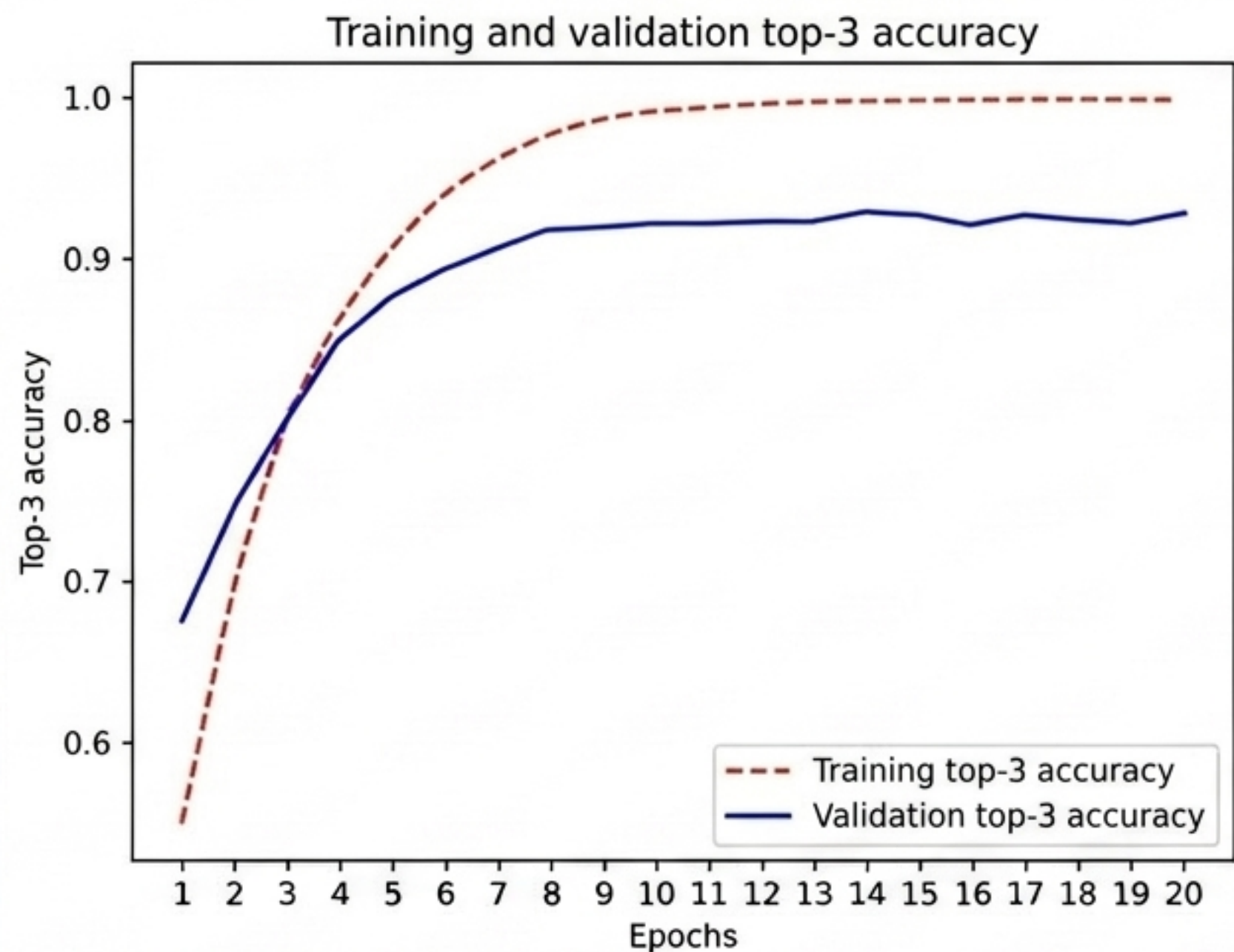
Đánh giá Dict hình Accuracy



Hình 4.7: Training và Validation Accuracy

- **Đỉnh Accuracy:** Đạt cao nhất khoảng 80% tại epoch thứ 9 (trước khi bị quá khớp).
- **Đường cơ sở (Baseline):** Xác suất chọn ngẫu nhiên đoán bừa theo tần suất tự nhiên của dữ liệu chỉ khoảng 19%.
- ✓ **Kết luận:** Đạt mức 80% trong bài toán 46 lớp là một kết quả tuyệt vời.

Nhưng 80% đã là con số thực?



Hình 4.8: Chỉ số Top-3 Accuracy

- **Thực tế tin tức:** Một bài báo có thể liên quan tới cả 2, 3 chủ đề đan xen.
- **Định nghĩa Top-3:** Đo lường xem tỷ lệ nhãn thật (Ground-truth) có nằm trong 3 phán đoán có xác suất cao nhất hay không.

✓ **Giá trị cốt lõi:** Nếu dự đoán đúng xếp thứ hai hoặc ba, đó vẫn là thông tin có giá trị. Chỉ số Top-3 này có thể lên tới >90%.



Bài toán Boston Housing

(Hồi quy Vô hướng - Scalar Regression)

Dự đoán giá nhà trung bình tại vùng ngoại ô Boston (thập niên 1970) dựa trên **13 đặc trưng** (tội phạm, thuế, số phòng...).

Bản chất Đầu ra



KHÔNG phải là xác suất hay lớp. Đầu ra là một con số vô hướng liên tục (Ví dụ: Dự đoán căn nhà giá 25.000 Đô la).

Thách thức Dữ liệu



Tập dữ liệu cực kỳ nhỏ, chỉ có 506 mẫu (404 mẫu huấn luyện, 102 mẫu kiểm tra).

Tiền xử lý: Chuẩn hóa Dữ liệu (Feature Scaling)

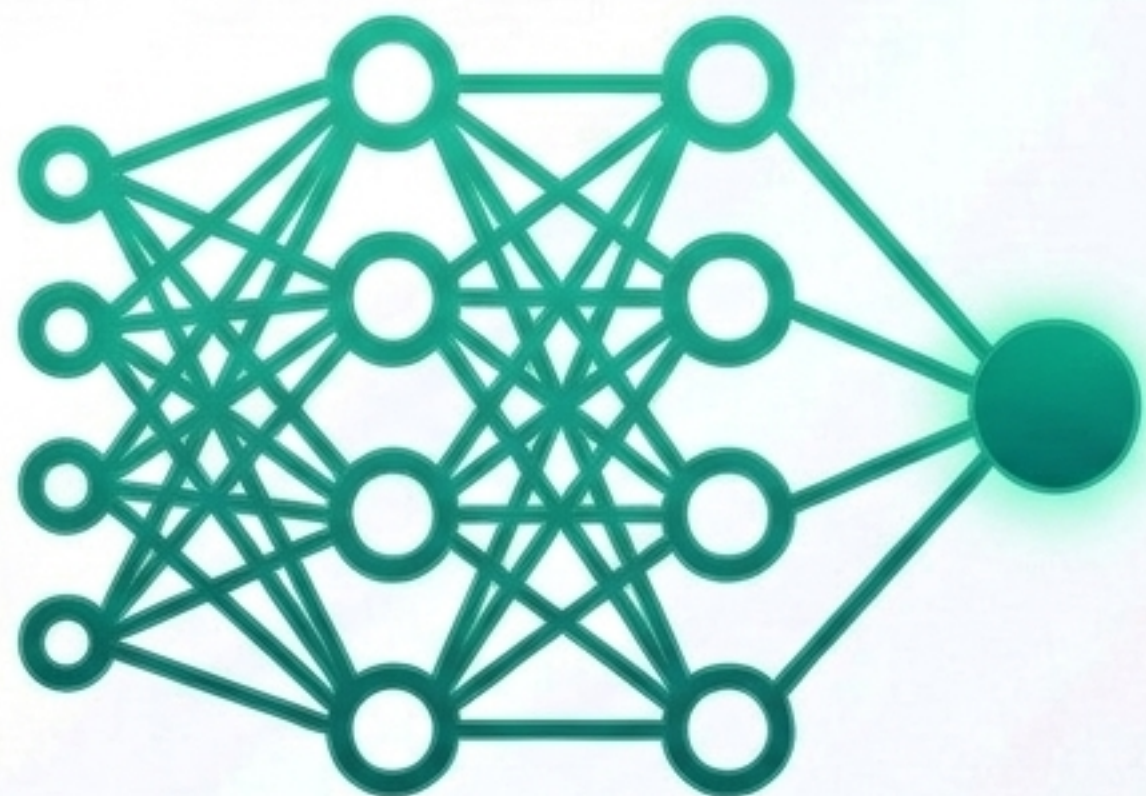


- **Các đặc trưng** (tội phạm, diện tích, tuổi nhà) có thang đo hoàn toàn khác nhau.
- **Đưa trực tiếp** vào mạng **nơ-ron** sẽ gây khó khăn cho việc hội tụ trọng số.
- **Công thức Chuẩn hóa (Normalization)**: Trừ đi giá trị trung bình và chia cho độ lệch chuẩn.

```
mean = train_data.mean(axis=0)
train_data -= mean
std = train_data.std(axis=0)
train_data /= std
```

```
# Cần phải chuẩn hóa tập test
theo trung bình của tập train
test_data -= mean
test_data /= std
```

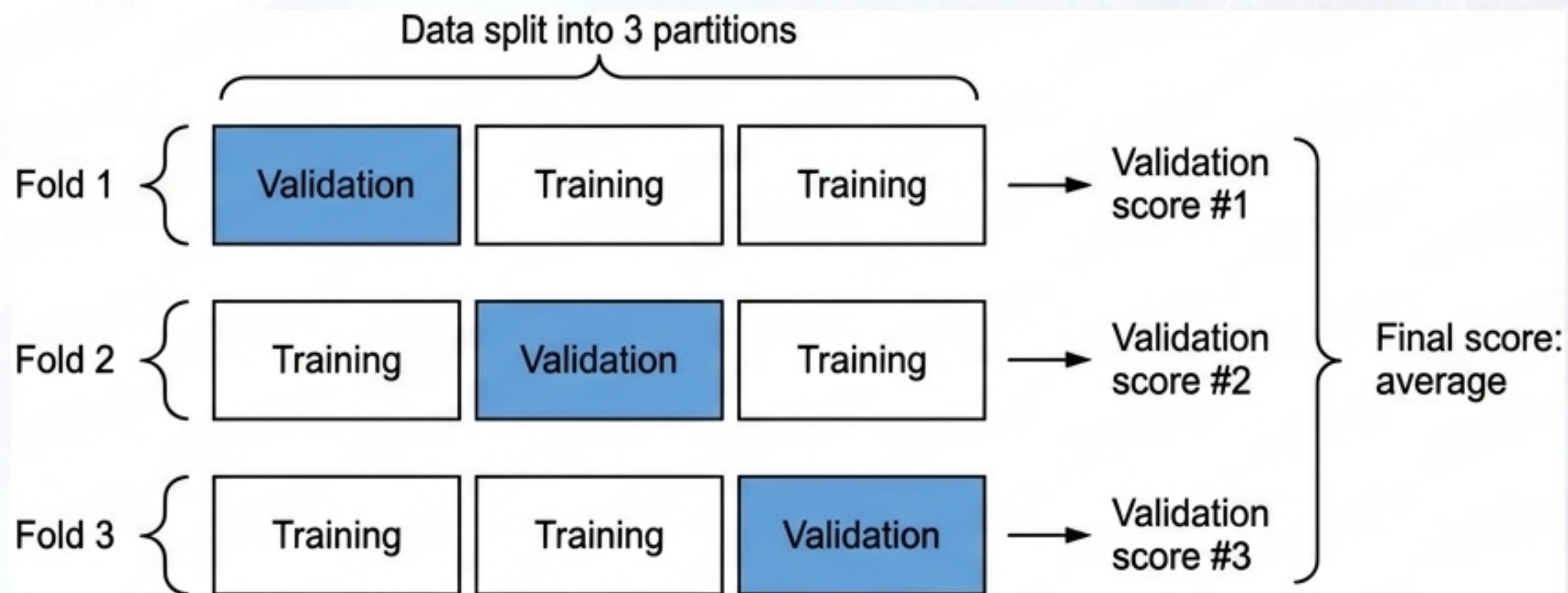
Xây dựng Mạng lưới Hồi quy



Do dữ liệu quá ít, mạng nơ-ron phải nhỏ (2 lớp ẩn, 64 units) để tránh Overfitting mạnh.

```
def build_model():  
    model = keras.Sequential([  
        layers.Dense(64, activation="relu"),  
        layers.Dense(64, activation="relu"),  
        layers.Dense(1) # Linear Layer - KHÔNG  
        có hàm kích hoạt  
    ])  
  
    # Loss MSE chuyển để tính lỗi hồi quy  
    # Metric MAE để con người dễ đọc (Mean  
    Absolute Error)  
    model.compile(optimizer="rmsprop",  
        loss="mse", metrics=["mae"])  
    return model
```

Đặc thù Dữ liệu Nhỏ & K-Fold Cross-Validation



Hình 4.9: Mô phỏng K-Fold Cross-Validation ($K=3$)

Vấn đề

Chỉ có 404 mẫu huấn luyện. Việc tách ngẫu nhiên một phần nhỏ (ví dụ 100 mẫu) làm tập Validation gây ra phương sai sai số lớn (phụ thuộc vào việc bốc trúng dữ liệu dễ hay khó).

Giải pháp K-Fold

Tách tập train làm K phần bằng nhau. Mỗi lần dùng 1 phần để validation, K-1 phần kia để train. Kết quả cuối cùng là trung bình của K vòng chạy.

Cài đặt K-Fold Validation



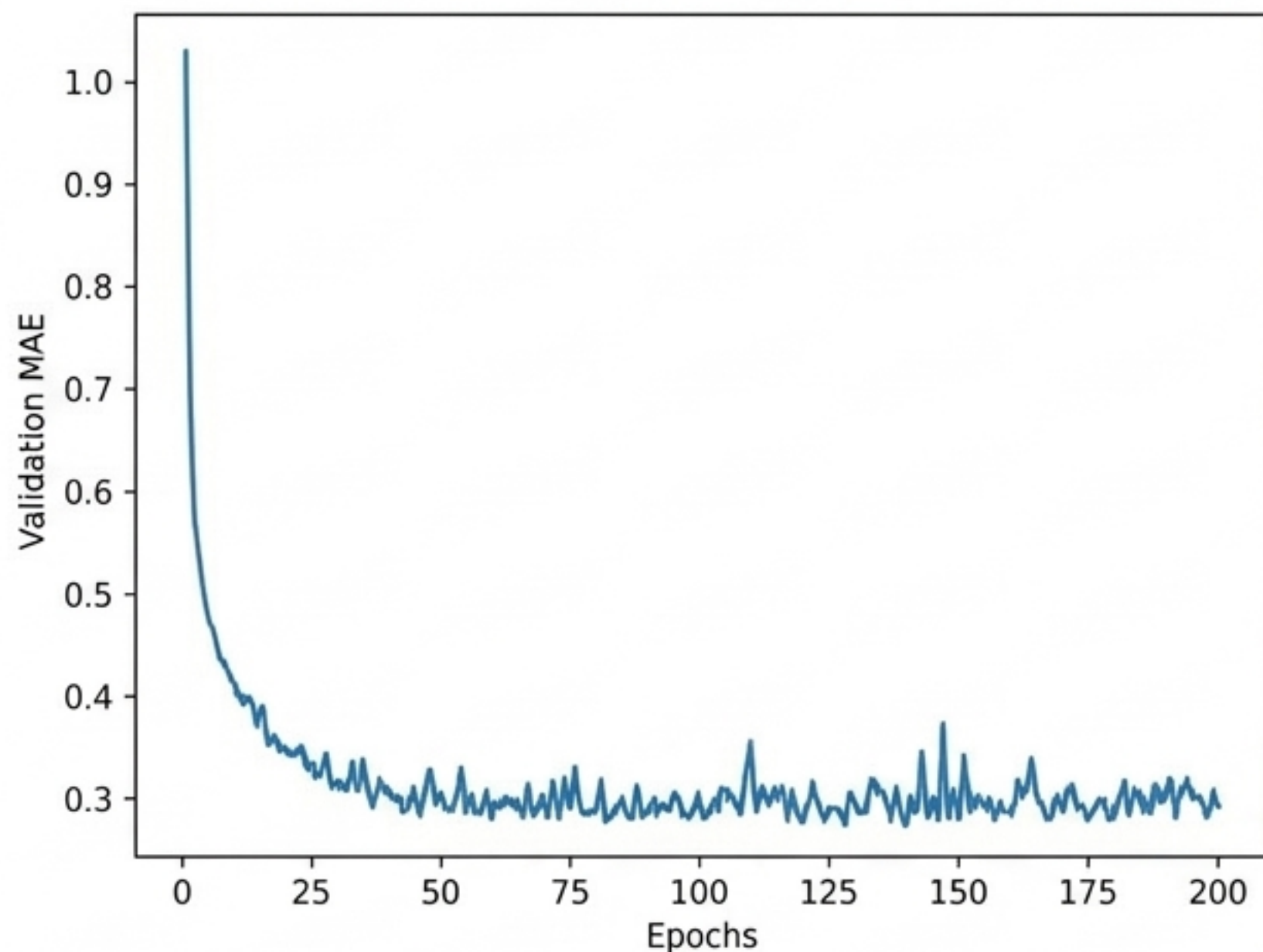
Mã nguồn thực thi vòng lặp tách dữ liệu thành K phần (ở đây $K=4$) và ghép các phần còn lại để làm tập huấn luyện.

```
k = 4
num_val_samples = len(train_data) // k
all_mae_histories = []

for i in range(k):
    val_data = train_data[i * num_val_samples: (i + 1) * num_val_samples]
    val_targets = train_targets[i * num_val_samples: (i + 1) * num_val_samples]

    # Ghep du lieu cac fold khac de lam tap huan luyen
    partial_train_data = np.concatenate([
        train_data[:i * num_val_samples],
        train_data[(i + 1) * num_val_samples:]], axis=0)
    # model = build_model()
    # model.fit(...)
```

Đánh giá Hiệu suất Hồi quy (MAE Plot)



Hình 4.10: Validation MAE qua 500 epochs

- 1. **Khác biệt Metric:** Trái với Accuracy (càng cao càng tốt), MAE (Mean Absolute Error) biểu thị cho sai số trung bình (càng thấp càng tốt).
- 2. **Ý nghĩa thực tế:** MAE = 0.5 tương đương mô hình dự đoán sai lệch khoảng 500 Đô la so với giá trị thực.
- 3. **Đặc điểm Đồ thị:** Trực quan hóa giá trị trung bình MAE qua K-folds trên 500 epochs. Đường đồ thị có độ rung (nhiều) rất cao ở những epochs đầu tiên trước khi dần ổn định.